Université Abdelmalek Essaadi

Faculté Polydisciplinaire à Larace

Département Informatique



Année universitaire 2025/2026

SMI – S6

Prof. Essaid EL HAJI

LLM BENCHMARK CONTAMINATION OBSERVATORY

Business Intelligence

Réalisé par : Mohamed Lgrindi

SMI 0213/23

GitHub https: https://github.com/mohamedlgrindi/llm-benchmark-observatory.git

1. Idée du projet

Les leaderboards de modèles de langage (LLMs) comme le HuggingFace Open LLM Leaderboard sont devenus la référence principale pour comparer les performances des modèles d'IA. Cependant, ils souffrent d'un problème structurel : la **contamination des données**. Ce phénomène se produit lorsque les données d'entraînement d'un modèle contiennent, intentionnellement ou non, les jeux de test des benchmarks utilisés pour l'évaluation. Le modèle obtient alors des scores artificiellement élevés, ce qui fausse les comparaisons.

Aucun outil public existant ne fournit un signal de détection de contamination au niveau modèle-benchmark. C'est ce manque que notre projet cherche à combler.

L'objectif est de construire un pipeline Business Intelligence complet qui :

- Collecte les données de scores de benchmarks depuis des sources réelles (HuggingFace Leaderboard v2)
- Les stocke dans un entrepôt de données structuré (PostgreSQL, schéma en étoile)
- Calcule un indicateur statistique appelé **suspicion_score** pour détecter les anomalies de performance
- Visualise les résultats dans un dashboard Power BI interactif

Le suspicion_score est notre contribution principale. C'est un z-score calculé par modèle et par benchmark :

suspicion_score = (score du modèle sur ce benchmark − moyenne de ses scores sur tous les benchmarks) / écart-type de ses scores

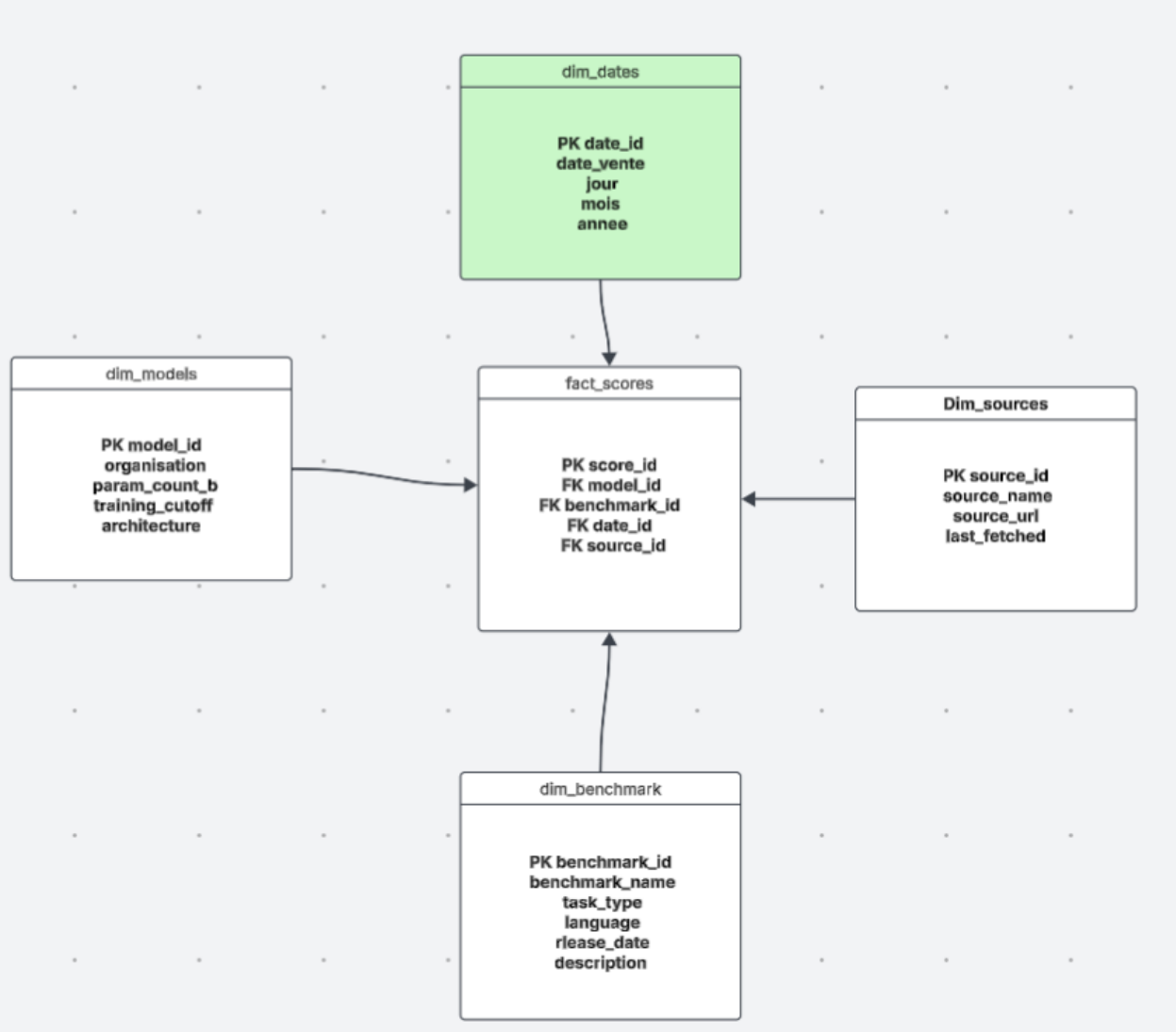
Un score élevé signifie que le modèle performe de manière anormalement bonne sur un benchmark spécifique par rapport à sa propre moyenne — un signal statistique cohérent avec une contamination des données d'entraînement.

```
Suspicion Score =
VAR AvgOld =
    CALCULATE(
        AVERAGE('public fact_scores'[score]),
        'public dim_benchmarks'[release_date] < DATE(2022, 1, 1)
    )
VAR AvgNew =
    CALCULATE(
        AVERAGE('public fact_scores'[score]),
        'public dim_benchmarks'[release_date] >= DATE(2022, 1, 1)
    )
RETURN
    IF(ISBLANK(AvgOld) || ISBLANK(AvgNew), BLANK(), AvgOld - AvgNew)
```

2. Schéma de la base de données

Pourquoi le schéma en étoile ?

Nous avons choisi un **schéma en étoile** (star schema) car c'est le modèle standard pour les workloads analytiques BI. Il permet d'exécuter des requêtes d'agrégation rapides sur de grandes tables de faits, tout en gardant les données de référence propres et réutilisables. Chaque décision de conception a été prise en tenant compte de la couche de reporting Power BI.



dim_models :

	model_id [PK] integer	model_name character varying (100)	organization character varying (100)	param_count_b double precision	training_cutoff date	architecture character varying (50)
1	1	0-hero/matter-0.2-7b-dpo	0-hero	7.242	[null]	MistralForCausalLM
2	2	01-ai/yi-1.5-34b	01-ai	34.389	[null]	LlamaForCausalLM
3	3	01-ai/yi-1.5-34b-32k	01-ai	34.389	[null]	LlamaForCausalLM
4	4	01-ai/yi-1.5-34b-chat	01-ai	34.389	[null]	LlamaForCausalLM
5	5	01-ai/yi-1.5-34b-chat-16k	01-ai	34.389	[null]	LlamaForCausalLM
6	6	01-ai/yi-1.5-6b	01-ai	6.061	[null]	LlamaForCausalLM
7	7	01-ai/yi-1.5-6b-chat	01-ai	6.061	[null]	LlamaForCausalLM

dim_benchmarks :

	benchmark_id [PK] integer	benchmark_name character varying (100)	task_type character varying (50)	language character varying (50)	release_date date	description text
1	1	ifeval	instruction following	english	2023-06-01	[null]
2	2	bbh	reasoning	english	2022-10-01	[null]
3	3	math_lvl5	mathematics	english	2021-11-01	[null]
4	4	gpqa	science	english	2023-11-01	[null]
5	5	musr	reasoning	english	2023-10-01	[null]
6	6	mmlu_pro	multitask	english	2023-08-01	[null]
7	7	mmlu	multitask	english	2020-01-01	[null]
8	8	humaneval	coding	english	2021-07-01	[null]
9	9	math	mathematics	english	2021-11-01	[null]
10	10	mt_bench	instruction following	english	2023-06-01	[null]

dim_dates :

	date_id [PK] integer	full_date date	year integer	month integer	quarter integer	weak integer	week integer
1	1	2026-03-...	2026	3	1	[null]	12

dim_sources :

	source_id [PK] integer	source_name character varying (100)	source_url text	last_fetched timestamp without time zone
1	1	HuggingFace	https://huggingface.co/datasets/open-llm-leaderboard/conte…	[null]
2	2	GitHub-LLM-Comparison	https://github.com/salttechno/LLM-Model-Comparison-2026	[null]

Table centrale — fact_scores :

	score_id [PK] integer	model_id integer	benchmark_id integer	date_id integer	source_id integer	score double precision
1	1	1	1	1	1	0.3303
2	2	1	2	1	1	0.3596
3	3	1	3	1	1	0.0144
4	4	1	4	1	1	0.2592
5	5	1	5	1	1	0.3814
6	6	1	6	1	1	0.1164
7	7	2	1	1	1	0.2841
8	8	2	2	1	1	0.5976

3. Processus ETL et code

Étape 1 — Extraction (Extract)

Nous avions initialement prévu d'utiliser **Talaxie** comme outil ETL visuel. Après plusieurs tentatives, Talaxie a été abandonné à cause d'incompatibilités persistantes avec le driver JDBC PostgreSQL. Nous avons remplacé cette approche par un **ETL Python natif** avec SQLAlchemy et pandas.

```
local_path = snapshot_download(repo_id="open-llm-leaderboard/contents", repo_type="dataset", local_dir="./hf_data")

print(f"Downloaded to: {local_path}")

print("\n Finding Parquet files")
```

```
print("Downloading LLM benchmark data from Github.")

url = "https://raw.githubusercontent.com/salttechno/LLM-Model-Comparison-2026/main/data/llm-model-comparison-2026.csv"
response = requests.get(url,timeout=30)
```

Étape 2 — Transformation (Transform)

```
    model_name = str(row.get("fullname","").lower().strip())
    if not model_name or model_name == "nan":
        continue

    if "/" in model_name:
        organization = model_name.split("/")[0]
    elif "-" in model_name:
        organization = model_name.split("-")[0]
    else:
        organization = "unknown"

    architecture =str(row.get("Architecture","unknown").strip())

    model_type = str(row.get("Type", "unknown").strip())

    param_count = row.get("#Params (B)", None)

    flagged = bool(row.get("Flagged", False))

    for raw_col, clean_name in available.items():

        score = row.get(raw_col,None)

        if score is None or str(score)== "nan":
            continue

        try:
            score = float(score)
        except (ValueError, TypeError):
            continue

        if score > 1.0:
            score = score / 100

        score = round(score, 4)
```

Étape 3 — Chargement (Load)

Les tables de dimension stables (`dim_benchmarks`, `dim_sources`) ont été peuplées manuellement via pgAdmin. Les tables dynamiques (`dim_models`, `fact_scores`) ont été chargées via SQLAlchemy :

```python
import pandas as pd
from sqlalchemy import create_engine
from datetime import datetime


engine = create_engine(
    "postgresql://postgres:Mohamedkali@localhost:5432/benchmark_dw"
)

print("Connected to PostgreSQL")



df_hf   = pd.read_csv("raw_hf.csv")
df_helm = pd.read_csv("raw_helm.csv")
df_all  = pd.concat([df_hf, df_helm], ignore_index=True)
print(f"Total rows loaded: {len(df_all)}")

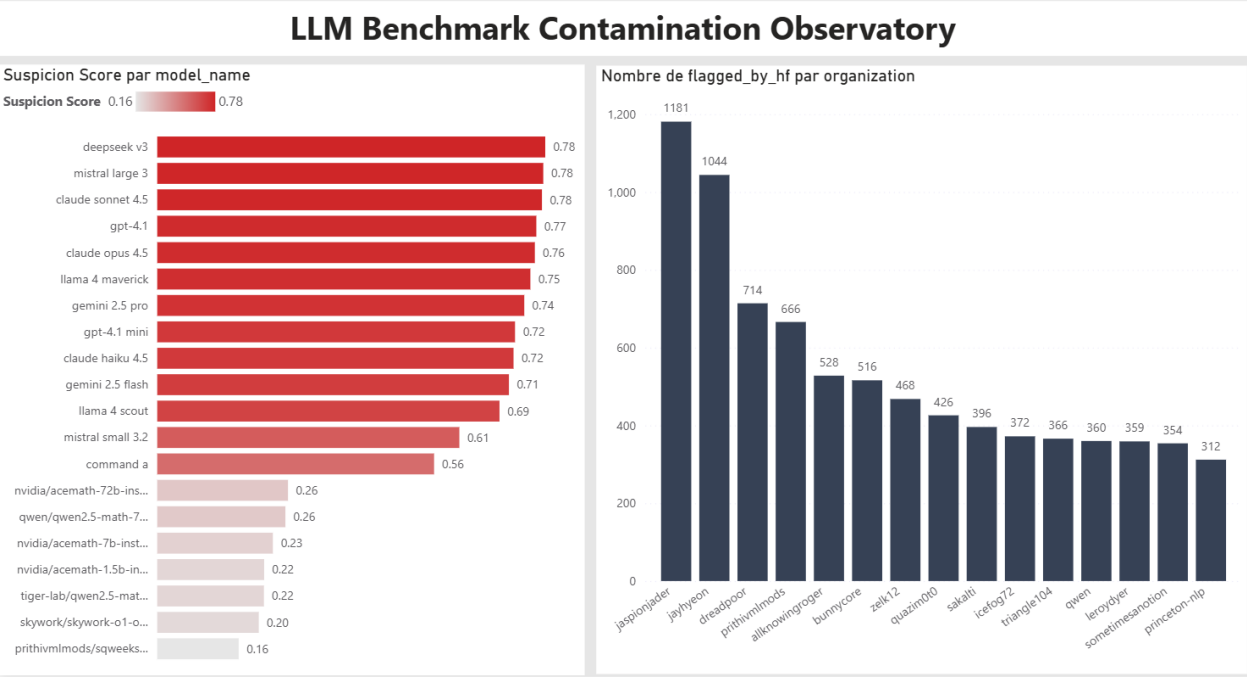


dim_models = df_all[[
    "model_name", "organization", "architecture", "param_count_b"
]].drop_duplicates(subset=["model_name"])
```

4. Visualisations Power BI

Power BI Desktop a été connecté à l'entrepôt PostgreSQL via le connecteur natif. Les relations du schéma en étoile ont été importées directement, avec `fact_scores` comme table centrale.

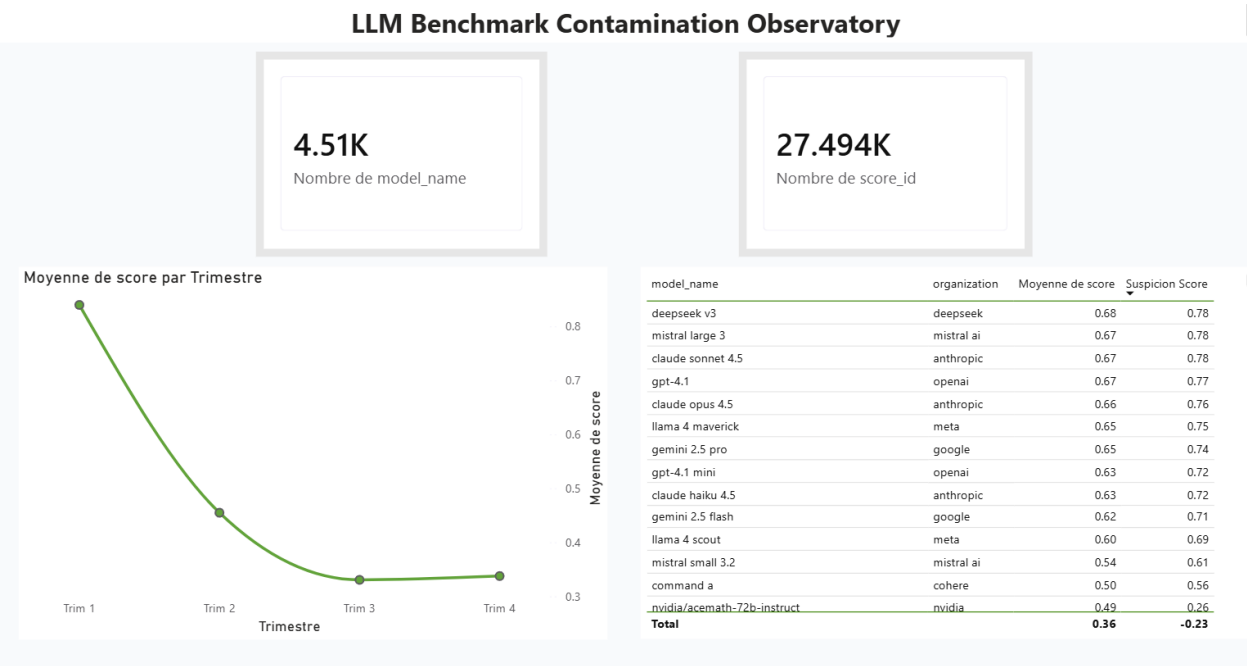
Page 1 — Vue d'ensemble de la contamination



Cette page contient deux visualisations principales. Le graphique en barres à gauche montre le **suspicion_score par modèle** : les scores vont de 0.16 à 0.78, avec DeepSeek v3, Mistral Large 3 et Claude Sonnet 4.5 en tête ex-æquo à 0.78. Le graphique à droite montre le **nombre de soumissions signalées par HuggingFace par organisation** : jaspionjader concentre 1 181 signalements, ce qui suggère un comportement de soumission organisé, non distribué.

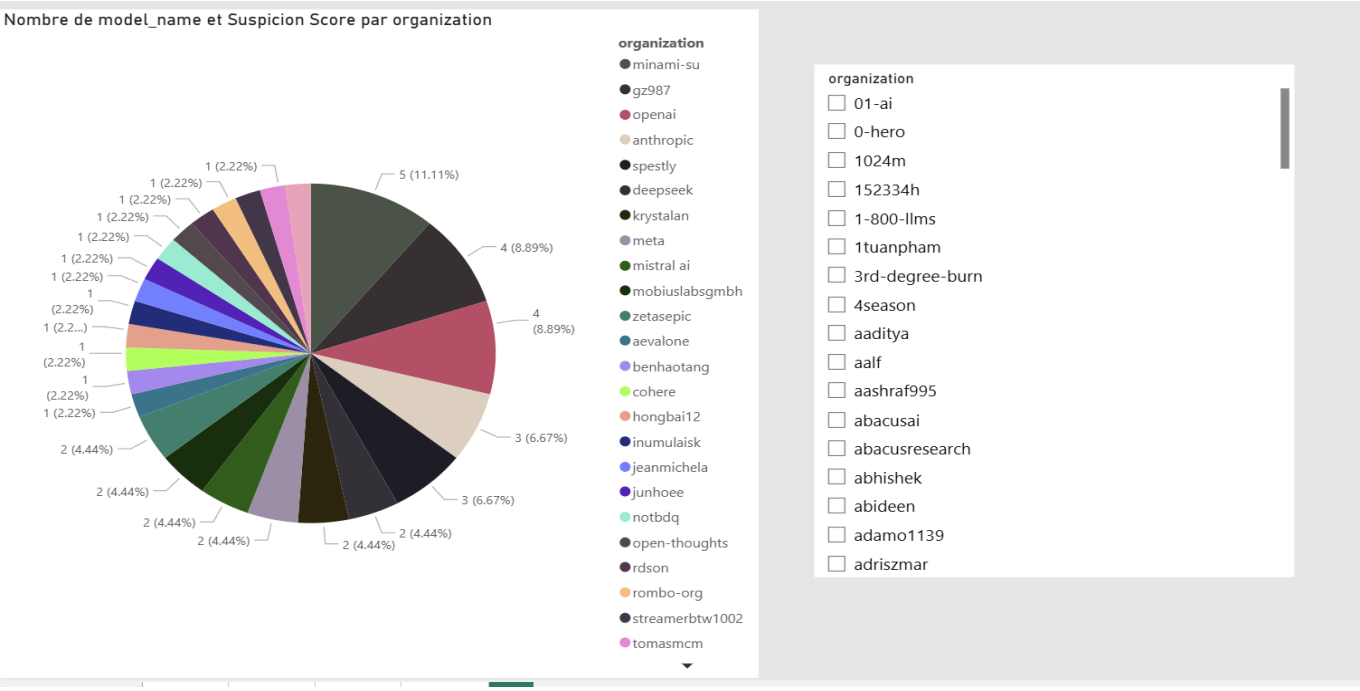
Conclusion : Les 13 premiers modèles (suspicion_score > 0.61) sont tous des modèles frontier de grands laboratoires (Anthropic, OpenAI, DeepSeek, Google, Meta, Mistral, Cohere).

Page 2 — Tendances des scores et table des modèles



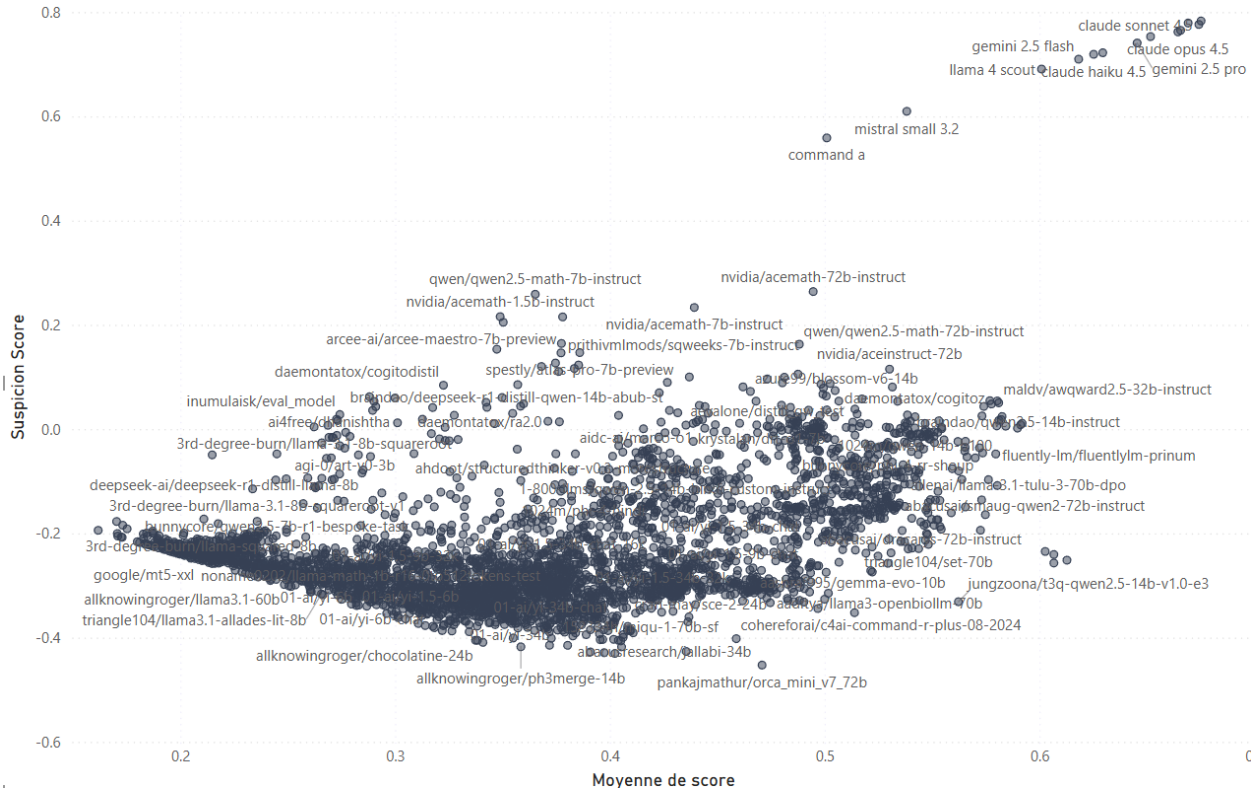
Les deux KPI cards confirment l'échelle du projet : **4 510 modèles uniques** et **27 494 enregistrements de scores**. Le graphique en courbe montre la **moyenne des scores par trimestre** : une chute nette de ~0.87 en T1 à ~0.36 en T3/T4. Cela reflète que le leaderboard est progressivement peuplé de modèles plus petits et moins compétitifs au fil du temps. La table de données permet de trier les modèles par organisation, score moyen et suspicion_score.

Page 3 — Analyse par organisation



Le graphique en secteurs montre la répartition des modèles par organisation parmi le segment à suspicion_score élevé. Les trois premières organisations sont minami-su (11.11%), gz987 (8.89%) et openai (8.89%). Le slicer à droite permet de filtrer dynamiquement toutes les visualisations par organisation.

Page 4 — Score moyen vs Suspicion Score



Ce nuage de points représente chaque modèle comme un point, avec en axe X sa moyenne de score sur tous les benchmarks, et en axe Y son suspicion_score.

Le résultat confirme visuellement l'argument central du projet : **les modèles les plus performants sont systématiquement les plus suspects**. On observe un cluster distinct dans le coin supérieur droit — Claude Sonnet 4.5, Claude Opus 4.5, Gemini 2.5 Pro, Gemini 2.5 Flash, Claude Haiku 4.5 — qui combinent à la fois un score moyen élevé (>0.6) et un suspicion_score proche de 0.8.

À l'inverse, la grande masse des modèles se concentre dans la zone centrale avec des scores moyens entre 0.2 et 0.5 et des suspicion_scores proches de zéro ou négatifs, ce qui indique des performances cohérentes sans anomalie détectable.

Ce graphique est la preuve visuelle que le suspicion_score n'est pas simplement un classement de performance — il capture une **anomalie relative** que les leaderboards traditionnels ne montrent pas.

Conclusion

Ce projet nous a permis de construire un pipeline Business Intelligence complet, de la collecte des données brutes jusqu'à la visualisation interactive.

Sur le plan technique, nous avons appris à concevoir un entrepôt de données avec un schéma en étoile, à manipuler des données réelles via l'API HuggingFace, et à calculer un indicateur statistique (le z-score) pour détecter des anomalies de performance.

Nous avons également fait face à plusieurs problèmes concrets :

- **Talaxie** n'a pas fonctionné avec PostgreSQL — nous avons dû le remplacer par un ETL Python natif

- **Papers With Code** n'existe plus en tant que source indépendante — nous avons pivoté vers un dataset GitHub

La leçon principale : dans un projet réel, les outils ne fonctionnent pas toujours comme prévu. La capacité à identifier le problème rapidement et à trouver une alternative est plus importante que le choix de l'outil initial.